

Use Ref Hook in React

the useRef hook in the react that allows us to persist value across renders without triggering the re-render, and also gives us the direct acces to DOM element.

(Refhook ek aisa tool hota hai jo kisi value ko yaad rakhta hai chahe wo kitani baar bhi render ho jaye . aur agar muze kisi html element ko directly acces karna hai to hum useRef ka use karke kr sakte hai)

Why and when do we use useRef?

1. Accessing DOM elements directly

Similar to document.getElementById, we use useRef to interact with DOM elements in a React way.

2. Storing mutable values that don't cause re-render

When we want to hold a value that can change, but we don't want the component to re-render when that value changes — useRef is the perfect choice.

3.

"useRef is mainly used for two things: referencing DOM elements and storing values that persist across renders without causing re-renders. It's lightweight and very useful when you need mutable containers in functional components."

My confusion → kabhi kabhi hum ref={} ka use he nahi kr rahe hai , to uska kya fayda

Confusion:

"Agar ref={...} karke kisi input pe nahi diya, to useRef diya kahan?"

Clear Answer:

useRef do kaam karta hai:

1. DOM access ke liye

Tab hum ref={myRef} lagate hain (jaise input ya div ke sath)

2. Non-DOM cheezein yaad rakhne ke liye (mutable variable jaisa)

Tab **kisi element ko ref={} se bind karna zaroori nahi hota.**

Hum bas us ref ke andar kuch value rakhte hain — jaise timer ID.

🏆 Tum ab confidently bol sakte ho:

- "useRef se value badalti hai, lekin React ka screen **tab tak nahi badlega** jab tak main kahun!"
- "useState bolta hai — bhai value badli? Main poora screen dobara paint kar deta hoon!"

check notes for clarification

Uncontrolled Components

uncontrolled component are react form element where the react does not controlled the form data. instead the data is handle by dom itself, using ref to access the value

```

import React, { useRef } from 'react'

const Uncontrolled = () => {

  const handleSubmit = (event) =>{
    event.preventDefault();
    let name = document.getElementById('name');
    let password = document.getElementById('password');
    console.log(name.value+password.value);
  }

  const nameref = useRef(null);
  const passref = useRef(null);
  const handleSubmitref = (event)=>{
    event.preventDefault();
    let name = nameref.current;
    let password = passref.current;
    console.log(name.value + password.value);
  }

  return (
    <div>
      <h3>with uncontrolled Component</h3>
      <form action="" onSubmit={handleSubmit}>
        <input type="text" id='name' placeholder='Enter name' />
        <input type="text" id='password' placeholder='Enter password' />
        <button>Submit</button>
      </form>

      <h3>with uncontrolled Component(use Ref)</h3>
      <form action="" onSubmit={handleSubmitref}>
        <input type="text" ref={nameref} placeholder='Enter name' />
        <input type="text" ref={passref} placeholder='Enter password' />
        <button>Submit</button>
      </form>
    </div>
  )
}

export default Uncontrolled

```

Pass function in component as props

why we need to pass function ?

→ when there is function in child component, and we are printing that component many times the function is also overwritten many times, best solution → make function in parent component and pass as a prop

passing event there is no such thing ?

→ There is **no special "event passing" syntax or concept** beyond standard event handling. React handles events via:

- Synthetic events (React's wrapper around native events)
- Props and callbacks

Forward Ref

jab hume kisi html element ka reference ek hi component ke andar dena padta hai to hum jo hai wo ref hook ka use karte hai , abh mano ki ref hook alag osme hai aur jisko refer karna hai wo alaga component mein hai , to hume use karna hai forwardref ka.

```
// import React from 'react'

// const UserInput = React.forwardRef((props, ref) => {
//   return (
//     <div>
//       <input type="text" ref={ref} />
//     </div>
//   );
// });

// export default UserInput;

// ----- after react 19 -----
import React from 'react'

const UserInput = (props) => {
  return (
    <div>
      <input type="text" ref={props.ref}/>
    </div>
  )
}

export default UserInput
```

```
// ForwardRef.js
import React, { useRef } from 'react'
import UserInput from './UserInput';

const ForwardRef = () => {
  const inputRef = useRef(null);

  const handleRef = () => {
    console.log("function called");
    inputRef.current.focus();
  }

  return (
    <div>
      <h3>Forward Ref</h3>
      <UserInput ref={inputRef} />
      <button onClick={handleRef}>Focus</button>
    </div>
  )
}

export default ForwardRef;
```
